

Project FORESIGHT

AI-Powered Demand Forecasting & Inventory Intelligence Platform

Complete Project Documentation

Assigned: 20 August 2026 | Submission Deadline: 20 September 2026

What This Project Does

- Predicts SKU-level product demand for the next 8 weeks using time-series forecasting.
- Automatically flags SKUs at risk of stockout or overstock based on forecast vs. current stock.
- Recommends exact reorder quantities for at-risk SKUs using a standard inventory formula.
- Presents everything on an interactive Streamlit dashboard with charts and colour-coded risk tables.
- Fully tested end-to-end, including a simulated fresh-clone run, with zero runtime errors.

What's Included

- Complete, working source code across 5 connected pipeline stages + dashboard
- A synthetic but realistic dataset (25 SKUs, 2 years of weekly data)
- One-command pipeline runner (run_pipeline.py)
- requirements.txt with exact tested versions
- README.md with setup instructions, ready for GitHub
- This documentation PDF, explaining every technology choice

1. How the Project Is Structured

FORESIGHT is built as five connected stages that each read from and write to a shared database, followed by a dashboard that only reads from that database. This means each stage can be tested, re-run, or explained independently, while **run_pipeline.py** ties all five together into a single command for a clean demo.

Stage	File	Input → Output
1. Data Cleaning	src/data_cleaning.py	Raw messy CSV → Clean CSV
2. Database Setup	src/database.py	Clean CSV → sales_history table
3. Forecasting	src/forecasting.py	sales_history → forecast_results table
4. Risk Detection	src/risk_detection.py	forecast_results + stock → risk_alerts table
5. Recommendations	src/recommendation.py	risk_alerts → recommendations table
Dashboard	dashboard/app.py	Reads all 4 tables → interactive UI

2. Technology Stack — What, and Why

Technology	Used For	Why This Choice
Python	Core language for the entire pipeline	One consistent language across data cleaning, modelling, database access and the dashboard; the standard choice for a data science project.
Pandas & NumPy	Data cleaning, transformation, feature prep	Industry-standard tabular data tools; used for deduplication, imputation, outlier handling and time-series reshaping.
statsmodels (ARIMA)	Demand forecasting model	ARIMA is a well-established, explainable time-series model suited to weekly sales data with trend and seasonality. Being explainable matters for a viva/interview: you can describe exactly what the model is doing, unlike a black-box model.
SQLite	Database (used in this build)	Zero-setup, file-based database with no server or credentials needed. Chosen specifically so the project runs identically on any machine during testing, grading, or a recorded demo, with no 'database connection failed' risk.
MySQL (documented path)	Production database (see docs/schema_mysql.sql)	The originally intended production database. All database access is isolated behind one function (get_connection()), so switching from SQLite to MySQL later only means changing that one function; every other file stays the same.
Streamlit	Interactive dashboard	Builds a genuinely interactive web dashboard in pure Python with no separate HTML/CSS/JS needed, ideal for a fast, demoable data science front-end.
Plotly	Charts inside the dashboard	Interactive (zoomable, hoverable) charts, which present a forecast far better in a live demo than a static image.

3. Stage-by-Stage Explanation

3.1 Dataset

A synthetic dataset covering 25 SKUs across 5 categories (Electronics, Grocery, Apparel, Home & Kitchen, Beauty), with 104 weeks (2 years) of sales and stock data per SKU, including real trend, seasonality, and promotion effects. It is deliberately messy in the same way a real retail export is: missing values, duplicate rows, inconsistent text casing, mixed date formats, outliers, and impossible entries like negative stock or zero price. This gives the cleaning stage genuine work to do rather than operating on already-clean data, which is more representative of a real project.

3.2 Data Cleaning — `src/data_cleaning.py`

What it does:

- Standardises text (casing, whitespace) and parses mixed date formats into one consistent format
- Removes exact duplicate rows
- Treats impossible values (negative stock, zero price) as missing, then imputes them
- Caps extreme outliers in Units_Sold using the IQR method, so a data-entry error doesn't distort a SKU's forecast
- Imputes remaining missing values per-SKU using that SKU's own median, so a low-selling item isn't filled with a high-selling item's typical value

Why this matters: forecasting models are sensitive to bad input — an unparsed date or a stray missing week can silently break a whole time series. Cleaning first means every later stage can trust the data it receives.

3.3 Database — `src/database.py`

What it does: Creates four tables and loads the cleaned data into the first one:

- **sales_history** — the cleaned weekly sales & stock records
- **forecast_results** — the forecasting stage's output
- **risk_alerts** — the risk detection stage's output
- **recommendations** — the recommendation engine's output

Why SQLite: it needs no server installation or login credentials, so the exact same code runs on any machine — yours, an evaluator's, or during a recorded demo — without any environment-specific setup that could fail on the day. All database access goes through one function, `get_connection()`, so upgrading to a real MySQL server later (schema provided in `docs/schema_mysql.sql`) is a one-function change, not a rewrite.

3.4 Forecasting — `src/forecasting.py`

Fits an ARIMA(1,1,1) model per SKU using `statsmodels`, forecasting 8 weeks ahead. Each model is first validated on a held-out window (its own last 8 weeks) to compute MAE (Mean Absolute Error) and RMSE (Root Mean Squared Error), so forecast quality is measured, not just assumed.

Reliability safeguard: if ARIMA fails to converge for a particular SKU (this can genuinely happen with short or irregular series), the code automatically falls back to a weighted moving average for that SKU only. This means one difficult SKU can never crash the whole pipeline — important for a live demo where a crash mid-run would be very visible.

3.5 Risk Detection — `src/risk_detection.py`

For each SKU, compares current stock against forecasted demand over that SKU's own lead-time window, and classifies it:

- **Stockout risk** — current stock is below forecasted demand for the lead-time window
- **Overstock risk** — current stock is more than 3× the forecasted demand for that window
- **Normal** — stock is within a healthy range

Each is further scored High/Medium/Low based on how far past the threshold it is. **Why rule-based rather than a machine-learning classifier:** a business user needs to be able to ask 'why is this flagged?' and get a clear answer. A transparent formula is more trustworthy, auditable, and easier to defend in a project viva than a black-box model for what is fundamentally a threshold decision.

3.6 Recommendation Engine — [src/recommendation.py](#)

Uses the standard inventory management formula for any SKU flagged as Stockout risk:

```
Reorder Quantity = Forecasted demand (lead time) + Safety stock - Current stock
```

Safety stock is set at 50% of the lead-time demand, as a buffer against demand variability so the SKU isn't reordered down to the bare minimum. Overstock SKUs instead get a 'hold off ordering' recommendation.

Why this matters: a forecast and a risk flag tell you there's a problem; this is the layer that tells you exactly what to do about it — the actual decision-support value of an 'inventory intelligence' platform, as opposed to a plain reporting tool.

3.7 Dashboard — [dashboard/app.py](#)

A Streamlit app showing: four KPI summary cards (total SKUs, stockout count, overstock count, SKUs needing reorder), a per-SKU actual-vs-forecast chart built with Plotly, a colour-coded risk table across all SKUs, and a forecast accuracy (MAE/RMSE) table. It only reads from the database — all computation already happened in the pipeline stages — which keeps the dashboard fast and simple.

4. Verified Results (From an Actual Pipeline Run)

Metric	Result
Rows in raw dataset	2,652 (25 SKUs × 104 weeks + injected duplicates)
Duplicate rows removed	52
Outliers capped (IQR method)	12
Missing values after cleaning	0
SKUs successfully forecasted	25 / 25 (all via ARIMA(1,1,1))
Average forecast MAE / RMSE	38.4 / 53.1 units per week
Risk mix detected	11 Normal, 8 Overstock, 6 Stockout
SKUs with an active reorder recommendation	6

How This Was Tested

- Every module (data_cleaning, database, forecasting, risk_detection, recommendation) was run individually and verified for correct output before being connected.
- The full pipeline (run_pipeline.py) was run start-to-finish with zero errors.
- The dashboard was tested using Streamlit's official AppTest framework, which actually executes the app script and reports any runtime exception — result: no exceptions.
- The entire project folder was copied to a separate location, all generated files (database, cleaned CSV) were deleted, and the whole pipeline plus dashboard were re-run from that clean copy to simulate exactly what happens after a fresh 'git clone' — it completed with zero errors.

5. How to Run This Project

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Run the full pipeline (cleans data, builds the database, forecasts, detects risk, generates recommendations):

```
python run_pipeline.py
```

3. Launch the dashboard:

```
streamlit run dashboard/app.py
```

Three commands, no external server, no API keys, no manual configuration required.

Putting This on GitHub

```
git init
git add .
git commit -m "Initial commit: FORESIGHT demand forecasting & inventory intelligence platform"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

The generated database file and cleaned CSV are excluded via .gitignore since they are fully reproducible — anyone cloning the repository just runs run_pipeline.py once to regenerate them.

Project Folder Structure

```
FORESIGHT/
  data/ – dataset generator, raw and (generated) clean CSVs, database file
  src/ – the five pipeline stage modules
  dashboard/app.py – the Streamlit dashboard
  docs/schema_mysql.sql – MySQL production schema reference
  run_pipeline.py – runs all five stages in order
  requirements.txt, README.md, .gitignore
```